TECHNICAL DOCUMENTATION

Real-Time Credit Card Fraud Detection API

A modular, high-performance machine learning microservice built with XGBoost and FastAPI for real-time transaction classification and risk scoring.

Executive Overview

This project implements an end-to-end Machine Learning pipeline capable of processing financial transaction data and assessing fraud probability in real time. Built upon an XGBoost classifier and exposed through an asynchronous FastAPI web service, the API automatically validates incoming payloads, aligns schema fields dynamically, imputes missing features, and outputs both binary risk flags and granular fraud probability metrics.

Key Features & Capabilities

- **Optimized ML Pipeline:** Implements an XGBoost model trained on PCA-transformed credit card transaction features, optimized for imbalanced classification tasks.
- **Asynchronous Web Engine:** Powered by FastAPI and Uvicorn for low-latency RESTful predictions.
- **Dynamic Feature Alignment:** Auto-populates missing dataset features with zero-imputation (0.0) and enforces exact column ordering required by Scikit-learn pipeline objects.
- **Interactive API Suite:** Auto-generated Swagger UI (`/docs`) and ReDoc specs for manual inspection and client integration testing.

Tech Stack Specification

Core Runtime: Python 3.13	**API Framework:** FastAPI, Uvicorn, Pydantic
ML Engine: XGBoost, Scikit-learn	**Data Processing:** Pandas, NumPy, Joblib

Project Repository Architecture

```
fraud_detection_xgboost/
├── app.py              # FastAPI application server & inference pipeline
├── train.py            # Model training script & artifact exporter
├── test_api.py         # HTTP test client script
├── model.joblib        # Serialized XGBoost pipeline artifact
├── requirements.txt    # Managed project dependencies
└── README.md           # Repository documentation
```

API Usage & Endpoints

1. Base Health Check

GET `/`

Verifies server status and model loading state.

2. Fraud Prediction Endpoint

POST `/predict`

Accepts a dictionary of transaction features and returns classification metrics.

Sample Request Payload:

```
{
  "features": {
    "V1": -1.359807,
    "V2": -0.072781,
    "V3": 2.536347,
    "V4": 1.378155,
    "Amount": 149.62
  }
}
```

Sample Response Output:

```
{
  "is_fraud": false,
  "fraud_probability": 0.0012
}
```

Local Setup & Execution Summary

To run this microservice locally, instantiate the virtual environment, install the requisite packages, and launch the Uvicorn ASGI server:

```
# Activate virtual environment
. env\Scripts ctivate

# Launch ASGI server
uvicorn app:app --reload
```